

INF 109 : TRAVAUX PRATIQUES — SÉANCE n° 2

Objectif : — La société *RapidRideTeleLogistics* est une jeune pousse fictive mais prometteuse qui souhaite se lancer sur le marché de la logistique des transports. Elle dispose d'une flotte de véhicules permettant de faire du transport de personnes (service de VTC). Chaque soir, elle doit déterminer quelles réservations doivent être confié à quel véhicule.

Dans ce TP, nous cherchons à fournir un planning optimal, c'est-à-dire un planning qui minimise le nombre de véhicules distincts qu'il faut mobiliser pour couvrir la totalité des demandes de réservation.

Consigne : — Pour chaque question, il est demandé d'écrire des tests pour justifier que la fonction fonctionne bien comme attendu. Chaque test sera introduit par l'instruction `assert` et sera conservé dans le fichier source afin d'être ré-exécuté à chaque modification de votre code.

Ce TP n'est pas noté ; aucun compte rendu n'est attendu.

2.1. Coordonnées géographiques

Nous approximations la Terre par une sphère de rayon $R_T = 6371$ km. Nous rappelons que la *distance orthodromique* entre deux points de latitude et longitude (λ_1, φ_1) pour le premier point et (λ_2, φ_2) pour le second point se calcule par la formule

$$2R_T \arcsin \sqrt{\sin^2 \left(\frac{\Delta\lambda}{2} \right) + \cos(\lambda_1) \cos(\lambda_2) \sin^2 \left(\frac{\Delta\varphi}{2} \right)}$$

où $\Delta\lambda$ et $\Delta\varphi$ représentent respectivement les différences de latitude et de longitude.

Nous souhaitons créer une classe permettant de représenter des coordonnées géographiques. En voici les premiers éléments. Une position est donnée par une latitude et une longitude (exprimées en degré d'angle).

```
31 from math import radians, cos, sin, asin, sqrt
32 from datetime import datetime, timedelta
33
```

```

34 class Position():
35     def __init__(self, lat : float, long : float):
36         self.lat = lat # Exprimée en degré
37         self.long = long # Exprimée en degré
38     def __str__(self):
39         return(f"({self.lat:.2f}N,{self.long:.2f}E)")

```

Question 1. — Enrichir la classe `Position` d'une constante `EARTH_RADIUS` dont on fixera la valeur.

Le rayon terrestre ne dépendant que de la terre, et non pas d'un point particulier, cette constante doit être définie au niveau de la classe et ne doit pas être répétée au niveau de chaque objet.

Pour répondre à la question suivante, on pourra utiliser les fonctions du module `math`. Attention, les fonctions trigonométriques prennent leur argument en radians.

Question 2. — Enrichir la classe `Position` d'une méthode `distance(self, other)` qui renvoie la distance orthodromique en km entre le point `self` et le point `other`. Si `P1` et `P2` sont deux positions, il doit être possible d'écrire simplement `P1.distance(P2)` pour calculer la distance.

La commande `timedelta(hours=25, minutes=100, seconds=300)`, provenant du module `datetime`, permet de définir une durée de 25 heures, 100 minutes et 300 secondes : les calculs et conversions sont faits automatiquement sans que l'utilisateur ait à réfléchir aux unités.

Question 3. — Enrichir la classe `Position` d'une méthode `time(self, other, speedinv)` qui renvoie le temps de parcours, de type `timedelta`, entre le point `self` et le point `other` à raison de `speedinv` secondes par kilomètre.

On ajoutera une valeur par défaut du paramètre d'entrée `speedinv` valant 180 s/km (soit 20 km/h) : il doit être possible d'écrire simplement `P1.time(P2)`.

2.2. Reservations de taxis

Nous souhaitons créer une classe permettant de représenter une demande de réservation d'un taxi. Un demande de réservation est constitué d'un nom de client, un temps de départ, une position géographique de départ et une position d'arrivée.

Nous proposons initialement

```

40 class Booking():
41     def __init__(self, name, depart_time, from_pos, to_pos):
42         if (not isinstance(name, str) or
43             not isinstance(depart_time, datetime) or
44             not isinstance(from_pos, Position) or
45             not isinstance(to_pos, Position)):
46             raise TypeError
47         self.name = name
48         self.depart_time = depart_time

```

```

49         self.from_pos = from_pos
50         self.to_pos = to_pos
51         self.taxi = None

```

Question 4. — Modifier le constructeur de la classe `Booking` pour que l'objet construit comporte un attribut supplémentaire, `eta`, qui correspond au temps d'arrivée estimé en se déplaçant à vol d'oiseau à une vitesse de 20 km/h.

2.3. Affectation grossière de taxis à des demandes de réservations

Nous nous proposons de coder une classe permettant de représenter l'attribution de demandes de réservations à une flotte de taxis.

Nous proposons initialement

```

52 class Planning():
53     def __init__(self):
54         self.bookings = [] # Liste de réservations
55         self.taxis = [] # Liste de listes
56     def add_booking(self, b : 'Booking'):
57         if isinstance(b, Booking):
58             self.bookings.append(b)
59         else:
60             raise(TypeError('Was expecting a booking.'))

```

Si `P` désigne un planning, nous supposons toujours que `P.taxis` est une liste de listes, que les taxis d'un planning `P` sont numérotés de 0 inclus à `len(P.taxis)` exclu et que pour tout indice $t \in \llbracket 0, \text{len}(P.taxis) \rrbracket$, la liste `P.taxis[t]` contient toutes les réservations couvertes par le taxi n° t .

Quand un taxi de n° t a été attribué à une réservation b , nous modifions systématiquement l'attribut `b.taxi` pour qu'il vale t . Autrement dit, on a toujours les invariants `assert(all([b in P.taxis[b.taxi] for b in P.bookings]))` et `assert(all(b.taxi == t for t in range(len(P.taxis)) for b in P.taxis[t]))`.

Question 5. — Enrichir la classe `Planning` d'une méthode `assign(self, t, b)` qui attribue le taxi n° t à la réservation b . Dans le cas où `len(self.taxis) ≤ t`, on commencera à compléter la liste des taxis.

Dans ce qui suit, on s'oblige à passer par la fonction `assign` pour créer une association entre un taxi et une réservation. Dans les questions qui suivent, nous passons en revue trois stratégies pour calculer une affectation complète de l'ensemble des demandes de réservations.

Question 6. — Enrichir la classe `Planning` d'une méthode `dummy_assignment(self)` qui attribue (ou ré-attribue) un taxi distinct à chacune des demandes de réservation présente dans `self.bookings`.

Nous cherchons à minimiser le nombre de taxis mis en service dans un planning donné.

Dans un premier temps, nous supposons que tout taxi qui a fini une réservation peut en reprendre une nouvelle n'importe où, du moment qu'elle démarre après un certain temps de latence, que nous fixons par défaut à 1h.

Question 6. — Enrichir la classe `Planning` d'une constante `lag_time` valant 1h et d'une méthode `greedy_assignment(self)` attribue (ou ré-attribue) un taxi à chacune des demandes de réservation présente dans `self.bookings` selon un principe glouton. On réfléchira à un ordre de traitement des événements concernant les réservations (à savoir, la mobilisation d'un taxi au début d'une réservation et la libération d'un taxi à la fin d'une réservation) pour que la stratégie programmée soit optimale. On pourra maintenir une collection de taxis libres entre deux événements.

2.4. Affectation optimale de taxis à des demandes de réservation

Dans cette section, nous voyons le problème de minimiser le nombre de taxis mis en service dans un planning donné comme un *problème de couplage dans un graphe biparti*.

Nous modélisons un ensemble de demandes de réservation par un graphe biparti $G = (V_0 \sqcup V_1, E)$.

- L'ensemble des sommets V_0 contient une copie de chaque demande de réservation.
- L'ensemble des sommets V_1 contient une autre copie de chaque demande de réservation.
- Il existe une arête $b_0b_1 \in E$ entre un sommet $b_0 \in V_0$ et un sommet $b_1 \in V_1$ si et seulement si un unique taxi dispose d'assez de temps entre la fin de la réservation b_0 et le début de la réservation b_1 pour se déplacer du point d'arrivée de la réservation b_0 au point de départ de la réservation b_1 .

Question 7. — Enrichir la classe `Planning` d'une méthode `_neighbours(self, b0)` qui renvoie la liste des voisins $b_1 \in V_1$ de $b_0 \in V_0$ dans le graphe biparti G .

Nous représentons un couplage $M \subseteq E$ dans le graphe G par des attributs `nextbooking` et `prevbooking` : pour toute arête $b_0b_1 \in M$, nous devons avoir `assert(b0.nextbooking == b1)` et `assert(b1.prevbooking == b0)`

Question 8. — Enrichir la classe `Planning` d'une méthode `_matching_init(self)` qui ajoute à toute réservation `b` des attributs `b.nextbooking` et `b.prevbooking` valant `None`.

Remarque : en python, un attribut peut être rajouté en cours de route ; il n'est pas nécessaire de le créer dès le départ dans le constructeur.

Question 9. — Enrichir la classe `Planning` d'une méthode `_match(self, b0, b1)` qui ajoute l'arête b_0b_1 au couplage.

Il pourra être prudent que le code teste les préconditions suivantes : b_0b_1 est réellement une arête et ni le sommet b_0 ni le sommet b_1 ne sont déjà couplés.

Pour toute affectation de taxis aux réservations, nous construisons un couplage $M \subseteq E$ formé des arêtes b_0b_1 telle qu'il existe un taxi dont le parcours traite b_0 puis b_1 immédiatement après.

Question 10. — Enrichir la classe `Planning` d'une méthode `_mark(self)` qui ajoute les attributs `nextbooking` et `prevbooking` à tous les sommets de G pour qu'ils correspondent au contenu de `self.taxis`.

Pour tout couplage $M \subseteq E$ dans le graphe G , on peut inversement construire une affectation dans laquelle deux demandes de réservations reliées sont couvertes par le même taxi.

Question 11. — Enrichir la classe `Planning` d'une méthode `assignment_from_marks(self)` qui, à partir des marques `nextbooking` et `prevbooking`, reconstruit une affectation entre taxis et réservations.

En notant $n = |V_0| = |V_1|$ le nombre de demandes de réservations et m le nombre d'arêtes dans le couplage, on constate que l'affectation utilise exactement $n - m$ taxis distincts.

Il y a ainsi une bijection entre couplages dans G d'une part et affectations réalisables entre demandes de réservations et flotte de taxi d'autre part. Pour trouver une affectation minimisant la flotte de taxi, il faut trouver un couplage de cardinal maximal.

Intéressons nous à présent à la construction d'un couplage maximum : il s'agit d'implémenter les idées du cours.

Question 12. — Enrichir la classe `Planning` d'une méthode `_find_augmenting(self)` qui construit une arborescence d'exploration du graphe biparti G en vue de trouver un chemin augmentant entre deux sommets non appariés.

Cette arborescence est retournée sous la forme d'un dictionnaire `parent` qui ne stocke que des associations dans le sens $V_1 \rightarrow V_0$ (qui correspondent à des arêtes de chemins augmentants qui n'appartiennent pas au couplage $M \subseteq E$; en revanche, il n'est pas nécessaire de dupliquer l'information concernant les arêtes de M , autrement dit, pas la peine d'avoir la partie $V_0 \rightarrow V_1$).

Ainsi, la valeur de retour de `_find_augmenting(self)` est, si possible, un sommet qui termine un chemin augmentant et le dictionnaire `parent` et vaut `None` sinon.

N'hésitez pas à rajouter des sous-fonctions qui décomposent le travail en parties plus simple à coder.

Question 13. — Enrichir la classe `Planning` d'une méthode `_unmatch(self, b0, b1)` qui retire l'arête b_0b_1 du couplage.

Il pourra être prudent que le code teste les préconditions suivantes : b_0b_1 est réellement une arête dans le couplage.

Question 14. — Enrichir la classe `Planning` d'une méthode `_augment(self, b1, parent)` qui transforme le couplage $M \subseteq E$ en le couplage $M \Delta P \subseteq E$ où P est un chemin augmentant se terminant en b_1 et repéré par l'arborescence `parent`.

Question 15. — Enrichir la classe `Planning` d'une méthode `optimal_assignment(self)` qui calcule une affectation optimale entre taxis et demandes de réservation.

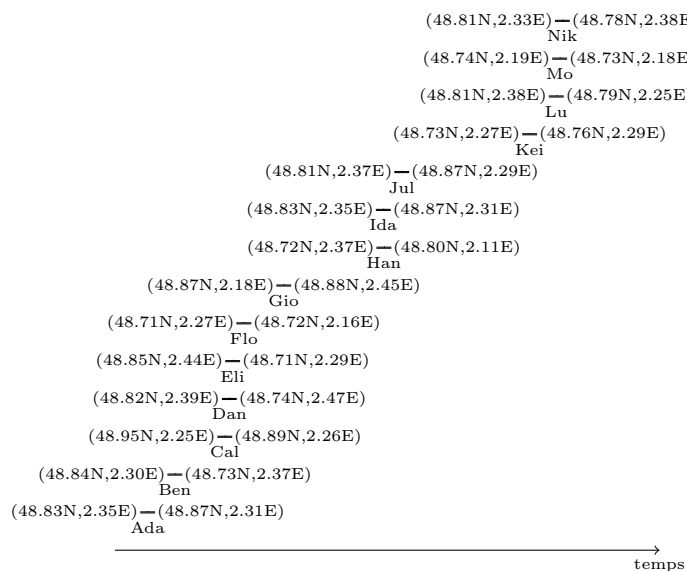


FIGURE 2.1. Exemple de demandes de réservation

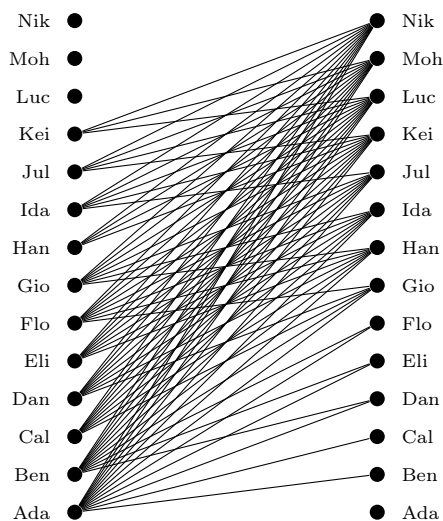


FIGURE 2.2. Graphe biparti indiquant les poursuites de réservation possibles